

AscensionAI Technical Write-Up

Generated: 2026-05-06

1. Project Basics

AscensionAI is a reinforcement learning project for training an AI agent to play Slay the Spire, currently focused on Ironclad. The project connects to a live Slay the Spire process through ModTheSpire, BaseMod, Communication Mod, and the bundled SpireComm Python interface. The Python side turns the game into a Gymnasium-style environment with a fixed observation vector, discrete action space, legal-action masking, shaped rewards, and PPO training.

The project is designed around long-running autonomous training rather than one-off scripted play. It supports behavior cloning warm starts, PPO fine-tuning, parallel rollout collection, offline PPO updates, greedy evaluation, passive game logging, progress plots, and a Windows control panel for launching and monitoring multiple Slay the Spire instances.

Current default training target:

- Game: Slay the Spire
- Character: Ironclad
- Ascension: 0 by default
- Environment interface: Communication Mod via SpireComm
- RL interface: Gymnasium-style Python wrapper
- Policy: actor-critic MLP trained with PPO
- Warm start: behavior cloning from a hand-coded heuristic
- Scaling: multiple rollout workers plus central offline trainer

2. High-Level Architecture

The project is built around this loop:

Slay the Spire

- <-> ModTheSpire / Communication Mod
- <-> SpireComm coordinator
- <-> Python agent scripts
- <-> observation encoder, action mask, reward tracker
- <-> PPO model

The live game exposes game state through Communication Mod. AscensionAI encodes that state into a 530-dimensional observation vector, computes a 134-action legal mask, chooses an action, sends the action back to the game, and records the resulting transition for training.

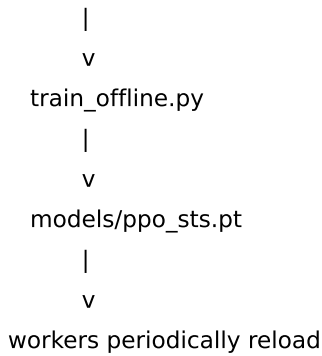
Parallel training separates data collection from model updates:

Worker 1 game -> rollout .npz

Worker 2 game -> rollout .npz

Worker 3 game -> rollout .npz

Worker 4 game -> rollout .npz



3. What The Project Uses

Core Python dependencies:

- numpy: numeric arrays and rollout serialization
- torch: neural network, PPO policy/value training
- gymnasium: environment-space conventions
- psutil: process monitoring and worker cleanup in the launcher
- matplotlib: training plot generation

Game/mod dependencies:

- Slay the Spire through Steam
- ModTheSpire
- BaseMod
- Communication Mod
- Super Fast Mode, recommended for faster live simulation
- SpireComm, bundled under external/spirecomm

Major ML/RL concepts used:

- Behavior cloning with cross-entropy imitation
- PPO fine-tuning
- GAE advantage estimation
- Actor-critic policy/value model
- Action masking for illegal game actions
- Dense reward shaping
- Parallel rollout collection
- Greedy evaluation without sampling

4. Project Achievements

AscensionAI has already moved beyond a small toy agent. The current project includes:

- A Gymnasium-style Slay the Spire environment wrapper.

- A 530-dimensional observation encoder covering player state, hand cards, deck state, monster state, powers, relics, potions, screen context, and map choices.
- A monster knowledge base for all 66 Slay the Spire monsters, including identity embeddings and behavioral flags such as enrages on skills, splits at low HP, scales strength, multi-attacker, retaliates, escapes, and spawns minions.
- A 134-action discrete action space covering card plays, targeted card plays, potions, event/card/map choices, proceed, leave, end turn, and no-op.
- Action masking so the policy only samples legal actions exposed by the current game state.
- Behavior cloning warm starts so PPO does not begin from a purely random policy.
- PPO training with clipped policy objective, value loss, conservative entropy bonus, target-KL early stopping, BC anchor loss, and GAE returns.
- Parallel rollout workers that write per-game transition files.
- A central offline trainer that filters stale rollout files, consumes fresh rollout batches, logs PPO diagnostics, and atomically saves updated checkpoints.
- A Windows control panel that launches workers, starts/stops training, tails logs, detects worker crashes, restarts instances, and tracks progress.
- Robustness patches for common Slay the Spire screen loops, including boss relic screens, combat rewards, map transitions, shops, card grids, Match and Keep, and events with disabled options.
- Fight tracking for elite and boss encounters, including terminal in-combat losses that would otherwise be missed.
- Progress tracking through CSV logs, fight logs, eval logs, fixed-seed evaluations, policy top-action logs, reward-correlation reports, and training plots.
- An archive workflow for clearing old logs, rollouts, and checkpoints before a clean experiment restart.

5. Current Limitations

The project is functional, but it is still an experimental RL system. Important limitations:

- The agent currently targets Ironclad only.
- The default launch path starts Ironclad at Ascension 0.
- Training uses a live game process, not a fast headless simulator. Throughput is limited by Slay the Spire simulation speed, animations, mod stability, and how many instances the machine can run.
- The model does not yet have strong proven win-rate performance. Learning progress should be judged through longer trend windows, greedy evaluation, and fight stats rather than short noisy samples.
- Some game surfaces are still supported by heuristics or fallback logic. The RL policy handles the main decision surfaces, but the system still uses mechanical handlers to keep runs moving.
- Shops, grid screens, events, and unusual modded states can still expose edge cases.
- The reward function is shaped, not a pure win/loss objective. This helps learning speed but can bias the model if shaping weights are wrong.
- PPO can forget behavior cloning habits if entropy, learning rate, or KL movement is too high. The current code reduces that risk but does not eliminate it.
- Workers collect data with the checkpoint they loaded at the time. Parallel workers may lag a few games behind the newest trainer checkpoint, so rollout files now carry checkpoint metadata and the trainer rejects stale files by default.

- Evaluation needs enough games to be meaningful. A 20-game greedy eval is useful for quick smoke testing but not enough to prove true performance.
- The project is currently Windows-oriented. The GUI, PowerShell launcher, process cleanup, Steam paths, and CommunicationMod config paths assume a Windows install.

6. How To Use The Project

6.1 Installation

Typical Windows setup:

```
git clone https://github.com/JustinoChan/AscensionAI.git
cd AscensionAI
python -m venv .venv
.\venv\Scripts\activate
pip install -r requirements.txt
pip install -e external/spirecomm
```

Install and enable these Slay the Spire mods:

- ModTheSpire
- BaseMod
- Communication Mod
- Super Fast Mode, recommended

Enable Fast Mode in Slay the Spire. If using Super Fast Mode, increase the speed slider to improve throughput.

6.2 GUI Usage

The main user interface is:

AscensionAI.pyw

Double-click it or run:

```
.\venv\Scripts\python.exe AscensionAI.pyw
```

The control panel exposes:

- Training mode
- Number of workers
- BC game count
- PPO game count
- Entropy coefficient
- Verbose logging toggle
- Start, Stop Now, and Finish && Stop controls

- Live logs for all workers and trainer
- Progress stats and training plot launcher

6.3 Command-Line Usage

The PowerShell launcher is:

```
.\launch_workers.ps1
```

Common patterns:

```
.\launch_workers.ps1 -Mode bc-ppo -BCGames 150
```

```
.\launch_workers.ps1 -NumWorkers 4
```

```
.\launch_workers.ps1 -Mode eval -Games 30
```

```
.\launch_workers.ps1 -Mode logger
```

The underlying Python scripts can also be launched directly through Communication Mod commands, but the control panel is the recommended path.

7. Ideal Order Of Running

For a clean restart from no useful model:

1. Archive or clear old active artifacts:

- models/ppo_sts.pt
- models/ppo_sts_bc.pt
- logs/*.csv
- logs/*.log
- rollouts_shared/*.npz

2. Run behavior cloning first:

- Recommended current range: 150 to 200 BC games.
- Purpose: teach the model basic card play, rewards, map choices, potion usage, target priority, and screen navigation.

3. Optionally run a small BC -> PPO continuation:

- 0 to 50 PPO games is enough as a sanity check.
- The main PPO learning should usually come from parallel workers.

4. Run parallel workers:

- Start with 4 workers if the machine is stable.
- Use entropy around 0.005 to 0.02 after BC. The default is 0.01.
- Keep verbose logging on for the first patched overnight run.

5. Let the offline trainer consume rollouts:

- Default batch: 5 completed rollout files per PPO update.
- Each update saves models/ppo_sts.pt.
- Workers reload the model periodically.

6. Evaluate greedily:

- Use Evaluation (Greedy) after 200 to 300 PPO games for early checks.
- For real comparisons, evaluate heuristic, BC-only, and PPO checkpoints on the same fixed seed list.
- Greedy eval removes sampling randomness, so it tests what the model currently prefers.

7. Plot trends:

- Use training_plot.png.
- Prefer rolling avg100 for floor/reward/win trend.
- Use avg25 only as a short-term indicator.
- Use lifetime average as a historical baseline.

8. What Each Code File Does

AscensionAI.pyw

Windows control panel. It launches Slay the Spire/ModTheSpire instances, writes CommunicationMod commands, starts workers and trainer, tails logs, tracks progress, handles Stop Now and Finish && Stop, detects crashed workers, and attempts relaunches.

launch_workers.ps1

PowerShell command-line launcher for BC, PPO, eval, logger, and parallel worker modes. Useful if the GUI is not desired.

scripts/obs_encoder.py

Converts the live game state into the fixed observation vector. Encodes player HP, energy, block, gold, floor, act, hand cards, card identity embeddings, deck profile, relic features, potion features, monster identity, intents, powers, move history, map choices, and screen context.

scripts/sts_gym_env.py

Defines the Gymnasium-style environment pieces: observation space, action space, action masks, flat-action to SpireComm action conversion, terminal detection, and reward shaping. This is where combat rewards, HP loss penalties, death/victory rewards, priority monster rewards, boss relic masking, event choice mapping, and legal action handling live.

scripts/ppo_model.py

Defines GameBuffer and PPOTrainer. The trainer owns the actor-critic network, action sampling, deterministic prediction, GAE advantage calculation, PPO clipped loss, value loss, entropy bonus, gradient clipping, checkpoint save, and checkpoint load.

It also reports PPO sanity metrics: approximate KL, clip fraction, explained variance, raw advantage mean/std, invalid action count, chosen-action probability, BC anchor loss, and whether target-KL early stopping triggered.

scripts/behavior_clone.py

Runs a heuristic policy to collect demonstration states and actions, saves the demo dataset, then trains the PPO network with supervised cross-entropy. It is the main BC warm-start script. Its heuristic includes combat target priority, card scoring, potion usage, event choices, card rewards, shops, rest sites, and map choices.

scripts/train_bc_ppo.py

End-to-end BC then PPO script. Phase 1 collects heuristic demos and trains the model. Phase 2 can continue with PPO games in the same session while keeping a small BC imitation anchor loss. It writes training stats and saves the BC checkpoint, BC demo dataset, and final PPO checkpoint.

scripts/train_ppo.py

Single-instance PPO trainer. One Slay the Spire process plays games and updates the model every configured number of games.

scripts/rollout_worker.py

Parallel data collector. A worker loads the current model, plays complete games, records RL transitions, writes one compressed .npz rollout per game with checkpoint metadata, and periodically reloads the latest model checkpoint.

scripts/train_offline.py

Central trainer for parallel workers. It watches rollouts_shared, waits for a batch of fresh rollout files, merges their transitions into one buffer, runs a PPO update, writes trainer stats, saves models/ppo_sts.pt, and deletes consumed files when launched with --delete-consumed. It rejects rollout files that are too many model updates behind and rejects legacy files without metadata unless explicitly allowed.

When it receives 5 rollout files by default:

1. It loads the first 5 ready .npz files.
2. It validates all arrays have matching lengths.
3. It merges observations, actions, rewards, dones, masks, log probabilities, and values.
4. It computes PPO advantages and returns.
5. It trains for the configured epochs and minibatches.
6. It appends a trainer row to training_stats.csv.
7. It saves the updated checkpoint.
8. It deletes or retires the consumed rollout files.

scripts/screen_handler.py

Shared screen-handling helpers. Handles card rewards, combat rewards, rest, events, boss relics, hand select, grids, shops, map choices, and mechanical proceed/cancel flows. It is used by workers, PPO training, BC, and evaluation to avoid duplicated screen logic.

scripts/game_data.py

Static card, relic, and potion knowledge used by the encoder and heuristic logic. This gives the model structured information instead of forcing it to infer every item from raw strings.

scripts/eval_model.py

Greedy evaluation runner. Loads the checkpoint, runs a fixed number of games with deterministic action selection, records win rate, average floor, reward, elite stats, and boss stats, but does not train. It can also run the heuristic baseline, use a fixed seed file, and log top-N model actions for policy inspection.

scripts/make_eval_seeds.py

Creates deterministic seed lists for fair heuristic-versus-BC-versus-PPO evaluation.

scripts/analyze_training_rewards.py

Reads training_stats.csv and reports correlations between shaped reward and actual outcomes such as final floor, act reached, wins, elite wins, and boss wins.

scripts/game_logger.py

Passive logger for recording game states while a human plays or while debugging CommunicationMod state transitions.

scripts/plot_training.py

Reads logs/training_stats.csv and generates training_plot.png. The current plot emphasizes rolling avg100, includes avg25 as a lighter short-term signal, and includes lifetime trend references for main episode metrics.

scripts/fight_tracker.py

Shared elite/boss fight tracker. Writes fight_stats.csv and legacy elite_stats.csv. Handles the important edge case where the terminal death state may still report in_combat=True.

scripts/analyze_trace.py

Utility for analyzing passive game logger traces.

external/spirecomm

Bundled SpireComm library that speaks Communication Mod's stdin/stdout protocol and exposes game-state/action objects to the agent.

9. Data And Artifacts

Important runtime directories:

- models/: stores ppo_sts.pt and ppo_sts_bc.pt.
- logs/: stores debug logs, training CSVs, eval CSVs, fight stats, and plots.
- rollouts_shared/: stores per-game .npz rollout files from parallel workers.

- archives/: stores old experiment snapshots when logs/models/rollouts are archived.
- seeds/: stores fixed seed lists for controlled greedy evaluation.
- docs/: stores this technical write-up and its PDF export.

Important generated files:

- models/ppo_sts.pt: main PPO checkpoint.
- models/ppo_sts_bc.pt: behavior cloning checkpoint.
- logs/training_stats.csv: game and trainer metrics.
- logs/fight_stats.csv: elite and boss fight outcomes.
- logs/eval_stats.csv: greedy evaluation results.
- logs/training_plot.png: visual training curves.
- rollouts_shared/*.npz: unconsumed game rollouts.
- seeds/eval_200.txt: default 200-seed controlled evaluation list.

10. What Is RL Versus Heuristic

The project uses both RL and heuristics. This is intentional.

RL policy decisions include:

- Combat card choices
- Combat target choices
- Potion choices when exposed to the policy
- Event choices
- Card reward choices
- Boss relic choices
- Rest-site choices
- Map choices
- Other decision-screen choices when delegated through the action mask

Heuristic or mechanical handling includes:

- Opening chests
- Confirming already-selected grids
- Proceeding through completed screens
- Recovering from known screen loops
- Fallback handling for empty or malformed choices
- Some shop and reward plumbing to avoid infinite loops
- Behavior cloning demonstration policy

The goal is not to remove every heuristic immediately. The current practical goal is to let RL own the decisions that matter while heuristics keep the live game process from stalling.

11. Reliability And Safety Mechanisms

Long-running modded Slay the Spire sessions are fragile. AscensionAI includes:

- Atomic model saves using temporary files and os.replace.

- Atomic rollout writes using temporary .npz files and os.replace.
- Worker heartbeat files.
- Worker crash detection and relaunch.
- ModTheSpire launcher fallback click handling.
- Stuck-state detection and bug_debug.log dumps.
- Finish && Stop for graceful worker shutdown.
- Stop Now for hard cleanup.
- Process cleanup for orphaned Slay the Spire Java processes.
- CSV compatibility handling for older cumulative step/transition rows.
- Per-fight elite/boss stats to avoid misleading aggregate metrics.

12. What The Project Is Missing Or Needs Next

Recommended next technical work:

- Routine controlled evaluation: heuristic, BC-only, and PPO checkpoints on the same fixed seed list every 250 to 500 PPO games.
- Checkpoint versioning: keep named checkpoints with training metadata instead of only ppo_sts.pt.
- Better experiment tracking: store hyperparameters, git commit, BC game count, PPO games, worker count, and entropy in a run manifest.
- More direct shop learning: gradually move more shop decisions into RL once screen stability is proven.
- More direct grid learning: handle special grids with stronger state representations or specialized actions.
- Stronger reward validation: compare reward shaping against actual win/floor outcomes over long runs and tune weights if correlation is weak.
- Saved policy-state snapshots: keep representative states for top-action inspection without needing a live eval run.
- Additional characters after Ironclad stabilizes.
- Cleaner cross-platform support if the project should run outside Windows.
- Optional headless or accelerated simulator path if a reliable non-live STS environment becomes available.

13. Recommended Metrics To Watch

Do not judge the model only by one short run. Useful metrics:

- Rolling avg100 final floor
- Lifetime average final floor
- Greedy evaluation average floor
- Greedy evaluation win rate
- Elite win rate
- Boss win rate
- Number of PPO updates
- Trainer-consumed transitions
- Policy entropy
- Value loss trend

- Approximate KL
- Clip fraction
- Explained variance
- BC anchor loss
- Stale/legacy rollout rejection count
- Reward/final-floor correlation
- Rollouts queued
- Worker crash/restart frequency
- Stuck-state dumps

For Slay the Spire, avg25 can be too noisy because one early death or one deep run can swing the trend line heavily. Avg100 is a better default learning signal.

14. Practical Fresh-Run Recommendation

For the current project state, a reasonable fresh run is:

1. Run 150 to 200 BC games.
2. Keep BC -> PPO PPO games at 0 to 50 unless you want a quick sanity check.
3. Start 4 parallel workers.
4. Use entropy around 0.005 to 0.02. The current default is 0.01.
5. Keep target KL at 0.03 unless PPO is barely changing or changing too violently.
6. Keep verbose logging on for the first long patched run.
7. Evaluate greedily after at least 200 to 300 PPO games, ideally on seeds/eval_200.txt.
8. Use avg100, lifetime averages, fight stats, reward correlation, and fixed-seed eval to decide whether the policy is improving.

15. Summary

AscensionAI is a live-game reinforcement learning system for Slay the Spire. It combines Communication Mod, SpireComm, Gymnasium-style environment wrapping, structured observation encoding, action masking, behavior cloning, PPO, parallel rollout collection, and long-run process supervision. Its current strength is that it can autonomously collect experience and improve from a BC warm start across multiple live game instances. Its current weakness is that live-game RL is slow, noisy, and fragile, so stability, logging, reward shaping, and evaluation discipline matter as much as the model architecture.